

บทที่ 5

Parallel Programming

การเขียนโปรแกรมแบบขนาน เป็นเทคนิคการเขียนโปรแกรมที่ใช้สำหรับการ execute operation พร้อมๆกัน ทั้งภายในเครื่องๆเดียว หรือข้ามระบบจำนวนหนึ่ง

ในการเขียนโปรแกรมแบบขนาน งานเดี่ยวหลายๆงานจะถูกแตกออกเป็นงานย่อยจำนวนหนึ่งที่สามารถถูกประมวลผลแยกจากกันอย่างเป็นอิสระ และถูกนำกลับมารวมกันเพื่อสร้างผลลัพธ์ที่ต้องการ

การเขียนโปรแกรมแบบขนานทำให้เราสามารถทำงานที่มีขนาดใหญ่และซับซ้อนให้เร็วขึ้นได้ จากการทำงาน (task) ขนาดใหญ่เราสามารถแบ่งย่อยเป็นงานที่มีขนาดเล็กกว่าและทำงานไปพร้อมๆกันได้ สิ่งที่สำคัญในการทำงานแบบขนานคือ

- การแบ่งงาน (task) ออกเป็นงานย่อยๆ
- การมอบหมายงานไปยังเครื่องที่ทำงานเพื่อทำงานพร้อมกัน
- การติดต่อสื่อสารเพื่อทำงานร่วมกันระหว่างเครื่องที่ทำงาน

การเขียนโปรแกรมแบบขนานจะเอาไปใช้แก้ปัญหา (solving) ต่างๆ เช่น การสร้างตึก, การทำงานในองค์กรขนาดใหญ่, การสร้างเครื่องจักรผลิตรถยนต์

5.1 ลักษณะของ parallel computer

ลักษณะของ parallel computer แบ่งออกเป็น 4 หมวดดังต่อไปนี้

5.1.1. Single Instruction, Single Data (SISD)

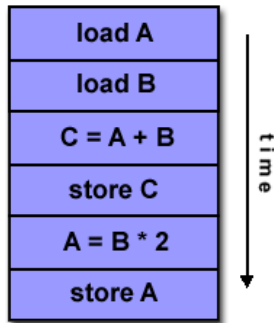
ไม่ใช่คอมพิวเตอร์แบบขนาน

ทำงานทีละคำสั่งเท่านั้นในแต่ละ clock cycle

มีการใช้งานข้อมูลอันเดียวเท่านั้นต่อหนึ่ง clock cycle

Deterministic execution

ประสิทธิภาพวัดเป็น MIPS (million of instructions per second) หรือ clock frequency เป็นMHz



รูปที่ 5-1 Single Instruction, Single Data (SISD)

5.1.2. Single Instruction, Multiple Data (SIMD)

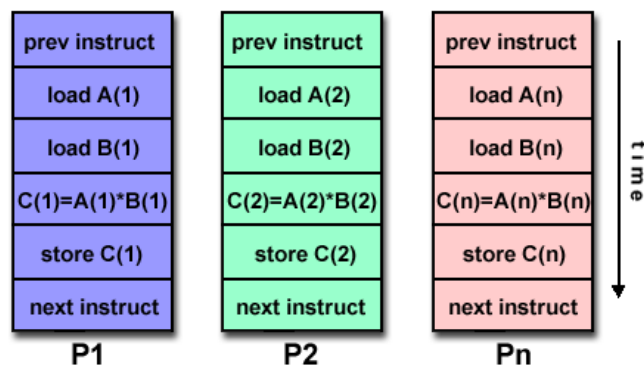
เป็นสถาปัตยกรรมแบบ von Neumann

คำสั่งแต่ละคำสั่งจะทำงานกับข้อมูลมากกว่าหนึ่งตัว

มี intermediate host ทำการ run โปรแกรมและ broadcast คำสั่งไปยังโปรเซสเซอร์ตัวอื่นๆ

Synchronous (lockstep)

ประสิทธิภาพวัดเป็น MFLOPS (million of floating point operations per second)



รูปที่ 5-2 Single Instruction ,Multiple Data (SIMD)

5.1.3. Multiple Instruction, Single Data (SIMD)

ในความเป็นจริงนั้นไม่ค่อยมีตัวอย่างของการทำงานแบบนี้

ตัวอย่างที่เป็นไปได้คือ filter ที่มีหลายความถี่ทำงานบน single signal stream และ cryptography algorithms หลายแบบซึ่งพยายามที่จะ crack รหัส

5.1.4. Multiple Instruction, Multiple Data (SIMD)

สามารถทำงานแบบขนานได้โดยการต่อโปรเซสเซอร์หลายๆตัวเข้าด้วยกัน

โปรเซสเซอร์แต่ละตัว execute คำสั่งของตัวเองไม่ขึ้นต่อโปรเซสเซอร์ตัวอื่นๆ

โปรเซสเซอร์แต่ละตัวทำงานกับข้อมูลที่แตกต่างกัน

สามารถ executions แบบ synchronous หรือ asynchronous, deterministic หรือ non-deterministic

มีปัญหา overhead ในการทำ load balancing ในการทำงานแบบ synchronization

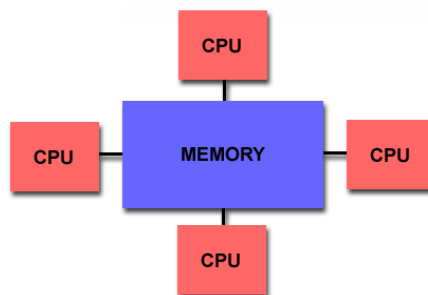
ยากต่อการเขียนโปรแกรม

เป็นรูปแบบของซูเปอร์คอมพิวเตอร์ในปัจจุบันและคอมพิวเตอร์แบบขนานที่เชื่อมต่อกันด้วย

เครือข่าย (กริด)

5.2. Parallel Computer Memory Architectures

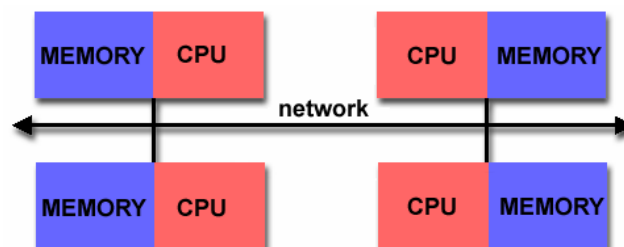
5.2.1. Shared Memory



รูปที่ 5-3 Shared Memory

โปรเซสเซอร์แต่ละตัวจะใช้หน่วยความจำเดียวกัน และ อ้างถึงหน่วยความจำโดยใช้ global address space ข้อมูลที่ถูกแก้ไขโดยโปรเซสเซอร์ตัวหนึ่งจะมีผลต่อโปรเซสเซอร์ตัวอื่นๆ แบบนี้จะง่ายต่อการพัฒนาโปรแกรม และมีความเร็วในการอ้างถึงข้อมูลสูง

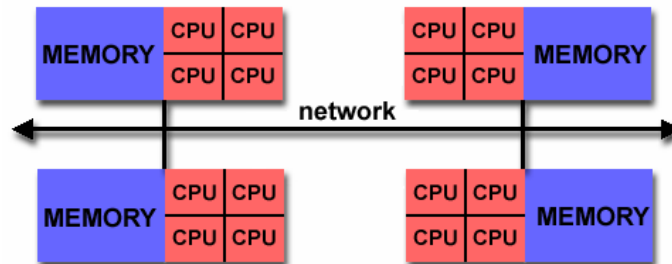
5.2.2. Distributed Memory



รูปที่ 5-4 Distributed Memory

โปรเซสเซอร์แต่ละตัวจะทำงานแบบอิสระต่อกันและมีหน่วยความจำเป็นของตนเอง การแชร์ข้อมูลจะกระทำโดยผ่านการติดต่อทางเครือข่ายและใช้การทำ message passing ซึ่งรูปแบบนี้จะไม่มีความ overhead ในการดูแล cache coherency

5.2.3. Hybrid Distributed-Shared Memory



รูปที่ 5-5 Hybrid Distributed-Shared Memory

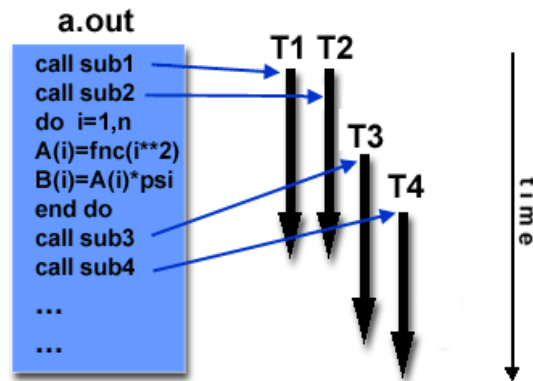
5.3 Parallel Programming Paradigms

5.3.1 Shared Memory Model

ในโมเดลของการเขียนโปรแกรมแบบแชร์นั้นงานจะถูกแชร์ address ที่ใช้ในการอ่านและ เขียน โดยจะมีกระบวนการต่างๆในการควบคุมการเข้าถึง shared memory เช่น locks และ semaphores ข้อได้เปรียบของโมเดลนี้จากมุมมองของโปรแกรมเมอร์คือไม่ต้องทำการติดต่อข้อมูลระหว่างงาน (task) ทำให้ง่ายในการพัฒนาโปรแกรม แต่จะมีปัญหาในการทำความเข้าใจและจัดการเรื่องของ data locality ในการ implement โมเดลนี้จะใช้ compiler เปลี่ยนตัวแปรของ user program ไปเป็นค่า actual memory addresses ที่เป็นแบบ global

5.3.2 Threads Model

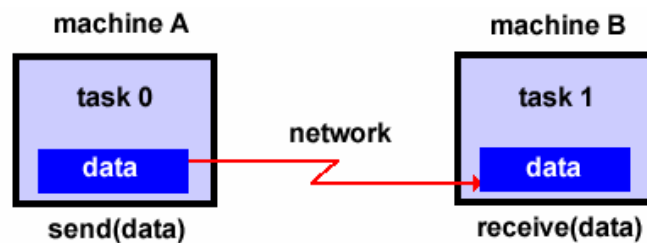
ในโมเดลแบบ thread โปรเซสหนึ่งโปรเซสจะสามารถมี path ที่ทำการ execution พร้อมกันได้หลาย path ยกตัวอย่างเช่นโปรแกรม a.out ถูกกำหนดให้ run โดยระบบปฏิบัติการ a.out จะถูกโหลดและจะถือครอง system และ user resources ที่จำเป็นในการ run ไว้ จากนั้น a.out จะสร้าง thread ที่สามารถรันโดยระบบปฏิบัติการพร้อมๆกัน ในแต่ละ thread จะมีข้อมูลของตัวเอง thread จะติดต่อสื่อสารกันผ่านทาง global memory ในการ implement โมเดลนี้จะใช้ POSIX Threads และ OpenMP



รูปที่ 5-6 Threads Model

5.3.3 Message Passing Model

โมเดลแบบนี้แต่ละงานจะมีหน่วยความจำของตัวเองในระหว่างการคำนวณ งานหลายๆงานสามารถอยู่บนเครื่องเดียวกันหรือต่างเครื่องกันก็ได้ งานจะแลกเปลี่ยนข้อมูลโดยการรับส่ง message ซึ่งอาศัยการทำงานร่วมกันของทั้งสองโปรเซสยกตัวอย่างเช่น โปรเซสหนึ่งกำลังส่งโปรเซสอีกโปรเซสก็ต้องทำการรับข้อมูล



รูปที่ 5-7 Message Passing Model

ในการ implement โมเดลนี้เราจะใช้ MPI (Message Passing Interface) ซึ่งเป็นมาตรฐานสำหรับการติดต่อแบบส่งข้อความ (Message Passing) นิยมใช้มากในการคำนวณแบบขนาน (Parallel Computing) และผู้ขาย (Vender) ทั่วไป MPI เป็นมาตรฐานที่ไม่ขึ้นกับแพลตฟอร์ม (Platform-Independent) และไม่ขึ้นกับภาษาที่ใช้เขียน (Language-Independent) สำหรับ library ของการส่ง

ข้อความ (Message-Passing Library) นั้นได้รับการพัฒนาโดย MPI Forum ซึ่งเป็นการร่วมมือกันระหว่างผู้ขายคอมพิวเตอร์แบบขนาน, ผู้เขียน library และผู้เชี่ยวชาญด้านแอปพลิเคชัน (Application Specialist) และ library ของ MPI นี้จะใช้ภาษา Fortran และ C เป็นหลัก

สาเหตุที่ทำให้ MPI เป็นที่นิยมนั้นเป็นเพราะว่ารายละเอียดของมันไม่ขึ้นกับผู้ขาย, ระบบปฏิบัติการ, ฮาร์ดแวร์ และภาษาที่ใช้ ซึ่งหมายความว่า โปรแกรมใดก็ตามที่เขียนโดยมีพื้นฐานมาจาก MPI (Based on

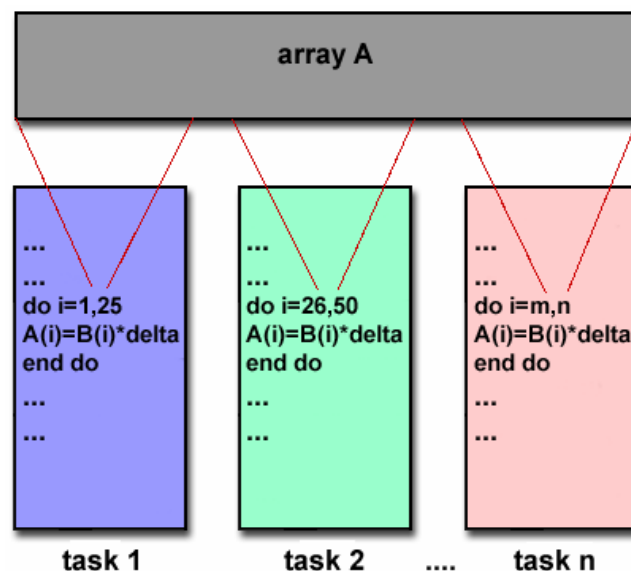
MPI) นั้นสามารถที่จะคอมไพล์(Compile)และ run ได้โดยไม่ต้องเปลี่ยน Source Code บน PC จากผู้ขาย หรือระบบปฏิบัติการใดๆ

นอกจาก MPI แล้วยังมี library ที่สำคัญคือ PVM ซึ่งเป็นโปรแกรมแบบขนานเช่นเดียวกัน โดยได้มีการพัฒนามาก่อน library ของ MPI แต่ปัจจุบันนี้นิยมใช้ library ของ MPI มากกว่า เนื่องจากมีฟังก์ชันการทำงานที่มากกว่า PVM และใช้ได้ง่ายกว่าด้วย

5.3.4 Data Parallel Model

โมเดลนี้จะมีคุณสมบัติดังนี้คือ งานแบบขนานส่วนใหญ่จะทำงานบนเซตของข้อมูล โดยข้อมูลจะถูกจัดโครงสร้างธรรมดาๆ เช่น array หรือ cube เซตของงานเก็บบนโครงสร้างข้อมูลเดียวกัน อย่างไรก็ตามแต่ละงานจะทำงานบน partition ที่แตกต่างกันบนโครงสร้างข้อมูลเดียวกัน (data structure)

ในสถาปัตยกรรมแบบ shared memory ทุกๆงานจะเข้าถึงข้อมูลโดยผ่าน global memory ส่วนสถาปัตยกรรมแบบ distributed memory ข้อมูลจะถูกแบ่งเป็น chunks ใน local memory ของแต่ละงาน



รูปที่ 5-8 Data Parallel Model

5.4 ขั้นตอนการสร้างโปรแกรมแบบขนาน

การเขียนโปรแกรมแบบขนานมีขั้นตอนวิธีการพัฒนาดังต่อไปนี้

5.4.1 ถ้าเราสร้างโปรแกรมขนาน (parallel program) จากโปรแกรมอนุกรม (serial program)

เราควรจะ Debug โปรแกรมอนุกรมให้เรียบร้อยก่อน จากนั้นเราควรศึกษา algorithm ด้วยว่าสามารถแก้ปัญหาโดยวิธี parallel programming ได้หรือไม่ ยกตัวอย่างเช่น

ตัวอย่าง โจทย์ที่แก้ได้โดย parallel programming:

คำนวณพลังงานศักย์ของ conformation หลายพันอันที่เป็นอิสระจากกันของแต่ละโมเลกุล เมื่อเสร็จแล้วหาค่า conformation ของพลังงานที่น้อยที่สุด

จากโจทย์ข้อมูลแต่ละอันเป็นอิสระต่อกันดังนั้นเราจึงสามารถคำนวณไปพร้อมๆกันได้จึงแก้ปัญหาโดยใช้ parallel programming ได้

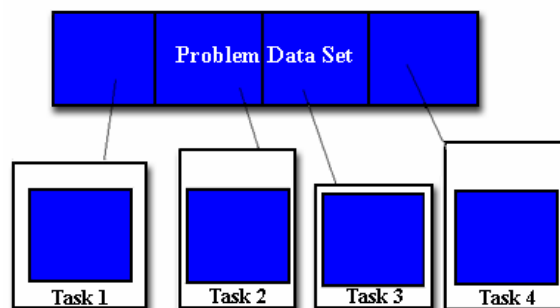
ตัวอย่าง โจทย์ที่แก้ไม่ได้ด้วย parallel programming:

การคำนวณ Fibonacci series (1,1,2,3,5,8,13,21,...) โดยใช้สูตรต่อไปนี้
$$F(k + 2) = F(k + 1) + F(k)$$

โจทย์นี้เราจะเห็นได้ว่าการที่เราจะบวกค่าได้ เราต้องอาศัยข้อมูลก่อนหน้านี้ ดังนั้นเราไม่สามารถจะทำการบวกพร้อมๆกันได้ จึงไม่สามารถใช้ parallel programming ในการแก้ปัญหาได้

5.4.2 ทำการระบุว่าส่วนไหนของโปรแกรมสามารถที่จะทำงานพร้อมกันได้

5.4.3 ทำการ Decompose ออกตามฟังก์ชันหรือข้อมูล



รูปที่ 5-9 Decompose Data Set

5.4.3.1 Function Decomposition (Functional Parallelism)

แบ่งปัญหาออกเป็นหลายๆส่วนเพื่อกระจายไปยังหลายๆ processor เพื่อทำการ execution พร้อมกัน ซึ่งแบบนี้จะเหมาะกับปัญหาที่มีโครงสร้างไม่คงที่และมีจำนวนการคำนวณที่แน่นอน

5.4.3.2 Domain Decomposition (Data Parallelism)

แบ่งขอบเขตข้อมูลของปัญหาและกระจายส่วนต่างๆไปยังหลายๆ processor เพื่อทำการ execution พร้อมกัน เหมาะแก่การใช้กับปัญหาซึ่ง

ข้อมูลมีค่าคงที่ (การ factor และแก้ไขปัญหา matrix ที่มีขนาดใหญ่หรือจำนวนการคำนวณที่จำกัด)

โครงสร้างข้อมูลที่เป็นแบบ dynamic ถูกรวมเป็น entity เดียวซึ่ง entity สามารถถูกแบ่งเป็น subset ย่อยๆได้ (ปัญหาแบบ multi-body ที่มีขนาดใหญ่)

ขอบเขตมีค่าจำกัดแต่การคำนวณในหลายๆส่วนของขอบเขตจะเป็นแบบ dynamic (fluid vortices models)

5.4.4 พัฒนาโค้ด (Code development)

5.4.5 compile, ทดสอบโปรแกรม, และ Debug

5.4.6 Optimization คือการวัดประสิทธิภาพ, กำหนดขอบเขตของปัญหา และปรับปรุงให้ดีขึ้น

5.5 สิ่งที่ต้องพิจารณาในการสร้างโปรแกรมแบบขนาน

5.5.1 Amdahl's Law

Amdahl's Law ระบุว่าความเร็วของโปรแกรมจะถูกกำหนดโดยส่วนของ code (P) ที่สามารถทำงานแบบขนานได้

$$\text{Speedup} = 1 / (1 - P)$$

ถ้าไม่มีส่วนของ code เลยที่สามารถทำงานแบบขนานแล้ว $P = 0$ แล้ว speedup = 1 (ไม่มี speedup) ถ้า code ทั้งหมดสามารถทำงานแบบขนานได้แล้ว $P = 1$ แล้ว speedup มีค่าเป็น infinity (ตามทฤษฎี)

ถ้า 50% ของ code สามารถทำงานแบบขนานได้แล้ว ค่า speedup สูงสุดจะเท่ากับ 2 ซึ่งหมายความว่า code จะ run เร็วขึ้นเป็น 2 เท่า

การหาจำนวนของ processor ที่ทำงานในส่วนที่ทำงานแบบขนานนั้น มีความสัมพันธ์ดังนี้

$$\text{Speedup} = 1 / ((P / N) + S)$$

เมื่อ P = parallel fraction (ส่วนที่ทำงานแบบขนาน)

N = จำนวนของโปรเซสเซอร์

S = serial fraction (ส่วนที่ทำงานแบบอนุกรม)

มีข้อจำกัดของ scalability ในการทำงานแบบขนาน ตัวอย่างเช่น เมื่อ

P = .50, .90 และ .99 (50%, 90% และ 99% ของ code ที่สามารถทำงานแบบขนานได้)

	speedup		
N	P = .50	P = .90	P = .99
10	1.82	5.26	9.17
100	1.98	9.17	50.25
1000	1.99	9.91	90.99
10000	1.99	9.91	99.02

ตารางที่ 5-1 ข้อจำกัดของ scalability ในการทำงานแบบขนาน

อย่างไรก็ตามสามารถแสดงประสิทธิภาพที่เพิ่มขึ้นโดยการเพิ่มขนาดของปัญหาตัวอย่างเช่น

2D Grid Calculations	85 วินาที	85%
Serial fraction	15 วินาที	15%

ตารางที่ 5-2 ประสิทธิภาพที่เพิ่มขึ้นโดยการเพิ่มขนาดของปัญหา

เราสามารถเพิ่มขนาดของปัญหาโดยการแบ่งครึ่ง grid points และ time step ซึ่งเป็นสัดส่วนโดยตรงกับ grid spacing ผลลัพธ์นี้เป็น 4 เท่าของจำนวน grid point (2 ส่วนในแต่ละทิศทาง) และ 2 เท่าของจำนวน time step การใช้เวลาเป็นดังนี้

2D Grid Calculations	680 วินาที	97.84%
Serial fraction	15 วินาที	2.16%

ตารางที่ 5-3 เพิ่มขนาดของปัญหาโดยการแบ่งครึ่ง grid points และ time step

ปัญหาซึ่งมีเปอร์เซ็นต์ของ parallel time มากขึ้นเกิดจากขนาดของมันที่มีความยืดหยุ่น (scalable) มากกว่าปัญหาที่มีเปอร์เซ็นต์ของ parallel time จำกัด

5.5.2 Load Balancing

Load balancing จะรับรองว่าการกระจายงานนั้นใช้เวลาในการ execution แบบขนานอย่างมีประสิทธิภาพที่สุด

ถ้างานถูกกระจายอย่างไม่สมดุลจะจัดการ โดยการหยุดรอเพื่อทำงานหนึ่งให้เสร็จก่อนในขณะที่งานอื่น ๆ มีสถานะว่าง

ประสิทธิภาพสามารถเพิ่มขึ้นได้ถ้างานถูกกระจายมากขึ้นอย่างเช่น ถ้ามีงานจำนวนมากซึ่งมีขนาดที่หลากหลาย มันจะเพิ่มประสิทธิภาพได้โดยการดูแล task pool และกระจายงานไปยังโปรเซสเซอร์เมื่อโปรเซสเซอร์แต่ละตัวทำงานเสร็จ

พิจารณาสภาพแวดล้อมที่ต่างกันเมื่อมีเครื่องจำนวนมากที่มีกำลังในการประมวลผลและ user load ที่ต่างกันทำงานในสภาพแวดล้อมเดียวกันคือมีโปรเซสเซอร์เหมือนกันโดยจะทำการ run งานหนึ่งงานต่อโปรเซสเซอร์หนึ่งตัว

5.5.3 Granularity

เพื่อที่จะให้โปรเซสเซอร์ที่ต่างกันทำงานร่วมกันในการแก้ปัญหาเดียวกัน ซึ่งจะต้องมีการกำหนดรูปแบบในการสื่อสารระหว่างโปรเซสเซอร์

อัตราส่วนระหว่างการประมวลผลและการสื่อสารจะถูกเรียกว่า granularity

granularity ที่มีประสิทธิภาพที่สุดนั้นจะขึ้นอยู่กับอัลกอริทึมและฮาร์ดแวร์ที่ใช้ทำงาน

ส่วนใหญ่ overhead ที่สัมพันธ์กับการสื่อสารและการ synchronization มีค่าสูงเมื่อเทียบกับความเร็วในการ execute ซึ่งเป็นข้อดีของ coarse granularity

5.5.3.1 Fine-grain parallelism

- งานทั้งหมดจะ execute คำสั่งจำนวนหนึ่งระหว่าง cycle ในการสื่อสาร
- อัตราส่วนระหว่างการประมวลผลกับการสื่อสารมีค่าต่ำ
- ใช้ประโยชน์จาก load balancing
- มี overhead ในการสื่อสารสูงและโอกาสในการเพิ่มประสิทธิภาพต่ำ
- ถ้า granularity ละเอียดยมากเกินไปจะต้องการ overhead ในการสื่อสารและการ synchronization ระหว่างงานใช้เวลามากกว่าเวลาในการประมวลผล

5.5.3.2 Coarse-grain parallelism

เป็นการประมวลผลแบบยาวซึ่งประกอบด้วยคำสั่ง (instruction) จำนวนมากระหว่างจุด synchronization ในการสื่อสาร

อัตราส่วนระหว่างการประมวลผลกับการสื่อสารมีค่าสูง

มีโอกาสนในการเพิ่มประสิทธิภาพสูง

ยากต่อการทำ load balancing อย่างมีประสิทธิภาพ

5.5.4 Data Dependency

- data dependency เกิดขึ้นเมื่อมีข้อมูลจำนวนมากใช้ storage location เดียวกัน
- สิ่งสำคัญของ dependency คือการป้องกันการทำให้ parallel execution อย่างสม่ำเสมอ
- ตัวอย่างเช่น

```
DO 500 J = MYSTART,MYEND
```

```
A( J) = A( J - 1 ) * 2.0
```

```
500 CONTINUE
```

จะเห็นว่า code นี้มี data dependency โดยต้องมีค่าที่ถูกประมวลผลสำหรับ A(J-1) ก่อนที่เราจะสามารถคำนวณ A(J) และถ้างานที่ 2 มี A(J) แล้วงานที่ 1 มี A(J-1) โดยค่าของ A(J) ขึ้นอยู่กับว่าเป็น Distributed memory หรือ Shared memory

Distributed memory

งานที่ 2 ได้รับค่าของ A(J-1) จากงานที่ 1

Shared memory

งานที่ 2 อ่าน A(J-1) ก่อนหรือหลังจากงานที่ 1 จะ update มันก็ได้

วิธีในการจัดการ data dependency

Distributed memory ทำการเชื่อมต่อข้อมูลที่ต้องการที่จุด synchronization

Shared memory ทำการ synchronize ในการอ่าน / เขียนระหว่างงานหลายๆงาน

5.5.5 Deadlock

- Deadlock อธิบายเงื่อนไขที่โปรเซส 2 โปรเซส หรือมากกว่านั้นกำลังรอ event หรือการสื่อสารจากโปรเซสอื่น
- ตัวอย่างที่ง่ายที่สุดถูกแสดงโดย 2 โปรเซสซึ่งถูกเขียนโปรแกรมเพื่ออ่าน / รับค่าจากอีกโปรเซสหนึ่งก่อนที่จะทำการเขียน / ส่ง

Task 1	Task 2
X = 4	Y = 8
SOURCE = TASK2 RECEIVE (SOURCE,Y)	SOURCE = TASK1 RECEIVE (SOURCE,X)
DEST = TASK2 SEND (DEST,X)	DEST = TASK1 SEND (DEST,Y)
Z = X + Y	Z = X + Y

ตารางที่ 5-4 ตัวอย่างการทำงานที่เกิด deadlock

วิธีการแก้ปัญหาวีธีหนึ่งคือการเปลี่ยนลำดับของ SEND และ RECEIVE ในส่วนหนึ่งของงาน

งานที่ 1	งานที่ 2
$X = 4$	$Y = 8$
SOURCE = TASK2 RECEIVE (SOURCE, Y)	DEST = TASK1 SEND (DEST, Y)
DEST = TASK2 SEND (DEST, X)	SOURCE = TASK1 RECEIVE (SOURCE, X)
$Z = X + Y$	$Z = X + Y$

ตารางที่ 5-5 การแก้ปัญหาลock โดยการเปลี่ยนลำดับของ SEND และ RECEIVE

วิธีแก้ปัญหาวีธีอีกวิธีหนึ่งคือการใช้ NON-BLOCKING message passing

5.5.6 รูปแบบในการสื่อสารและแบนด์วิดท์

ในบางปัญหานั้นการเพิ่มจำนวนของโปรเซสเซอร์จะลด execution time ในการประมวลผล แต่เพิ่ม execution time ในการสื่อสาร

เวลาที่ใช้ในการสื่อสารขึ้นอยู่กับ bandwidth ที่ใช้ในการสื่อสาร

ตัวอย่างเช่น เวลา (t) ที่ต้องการเพื่อส่ง W words ระหว่างโปรเซสเซอร์ 2 ตัวคือ

$$t = L + W/B$$

โดยที่ $L = \text{latency}$

$B = \text{hardware bitstream rate}$ เป็น word ต่อวินาที

Latency นั้นสามารถคิดว่าเป็นเวลาที่ต้องการเพื่อส่ง zero byte message

- รูปแบบการสื่อสารจะส่งผลกระทบต่ออัตราส่วนของการประมวลผลต่อการสื่อสาร

ตัวอย่างเช่น การสื่อสารแบบ gather-scatter ระหว่างโปรเซสเซอร์ตัวหนึ่งกับโปรเซสเซอร์อื่น

N ตัวจะได้รับผลกระทบจากการเพิ่ม latency มากกว่าโปรเซสเซอร์ N ตัวที่เชื่อมต่อเพียงแค่ว่ากับ neighbor ที่ใกล้ที่สุด

5.5.7 I/O Patterns

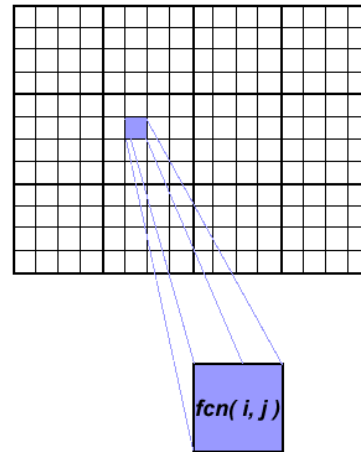
- I/O operation มักจะถูกมองว่าเป็นตัวขัดขวางการทำงานแบบขนาน
- ระบบ I/O แบบขนานนั้นยังไม่ถูก define และยังไม่มีการใช้งานในปัจจุบัน

- ในสภาพแวดล้อมที่ processor ทั้งหมดเห็น filesystem เดียวกันนั้นการเขียนจะทำให้เกิดการ overwrite file
- ความสามารถของ fileserver ส่งผลกระทบต่อการอ่านคือจะทำการร้องขอในการอ่านหลายๆ อันในเวลาเดียวกัน
- I/O ซึ่งต้องถูกดำเนินการบนเครือข่าย (non-local) อาจจะทำให้เกิดปัญหา bottleneck
- วิธีการอื่นๆ ได้แก่
 - ลด I/O ทั้งหมดเท่าที่จะเป็นไปได้
 - กำหนดขอบเขตของ I/O ให้เป็นส่วนที่ทำงานแบบ serial โดยเฉพาะ ตัวอย่างเช่น งานที่ 1 สามารถอ่าน input file และสื่อสารข้อมูลที่ต้องการให้กับงานอื่นๆ และงานที่ 1 สามารถทำการเขียนหลังจากได้รับข้อมูลที่ต้องการจากงานอื่นทั้งหมดแล้ว
 - สร้าง filename เฉพาะสำหรับ input/output file ของแต่ละงาน
 - สำหรับระบบหน่วยความจำแบบกระจายที่มีการแชร์ filesystem จะทำงาน I/O ใน filesystem ที่เป็นแบบ local และ non-shared ตัวอย่างเช่น แต่ละโปรเซสเซอร์ สามารถใช้ /tmp filesystem ซึ่งมีประสิทธิภาพมากกว่าการทำงาน I/O บนเครือข่ายไปยัง home directory ของเครื่องแต่ละเครื่อง

```

do j = 1,n
do i = 1,n
  a(i,j) = fcn(i,j)
end do
end do

```



รูปที่ 5-10 Array Processing

5.6 ตัวอย่างโปรแกรมแบบขนาน (Parallel Example)

5.6.1 Array Processing

ตัวอย่างนี้จะแสดงการคำนวณบนอาร์เรย์ 2 มิติ ซึ่งข้อมูลในแต่ละส่วนไม่ขึ้นต่อกัน ถ้าเป็นการคำนวณจากโปรแกรมแบบอนุกรม (serial program) จะมี source code ดังนี้

5.6.1.1 Array Processing Parallel Solution 1

find out if I am MASTER or WORKER

if I am MASTER

initialize the array

send each WORKER info on part of array it owns

send each WORKER its portion of initial array

receive from each WORKER results

```

else if I am WORKER

    receive from MASTER info on part of array I own
    receive from MASTER my portion of initial array

    # calculate my portion of array
    do j = my first column,my last column
    do i = 1,n
        a(i,j) = fcn(i,j)
    end do
    end do

    send MASTER results

endif

```

จาก Source Code ข้างต้นจะมีตัวประมวลผลอยู่สองประเภทคือ Master และ Worker ซึ่งตัว master จะเป็นคนแจกจ่ายขอบเขตของงานไปยัง worker และรอรับผลลัพธ์ในการทำงานกลับมา ส่วน worker จะนำเอางานที่ได้จาก master มาคำนวณแล้วส่งผลลัพธ์กลับไป

5.6.1.2 Array Processing Parallel Solution 2 :Pool of Tasks

```

find out if I am MASTER or WORKER

if I am MASTER

    do until no more jobs
        send to WORKER next job
        receive results from WORKER
    end do
endif

```

```
end do

tell WORKER no more jobs

else if I am WORKER

do until no more jobs
    receive from MASTER next job

    calculate array element:  $a(i,j) = fcn(i,j)$ 

    send results to MASTER
end do

endif
```

จากแบบที่ 1 เราจะเจอปัญหาในเรื่องของ load balancing เนื่องจากเรามีการแจกจ่ายงานแบบ static ดังนั้นถ้าเครื่อง worker ตัวไหนสามารถทำงานได้เร็วกว่า ก็จะมีเวลาว่างเหลือ ซึ่งการแก้ปัญหานี้เราจะใช้วิธี Pool of Tasks จะเป็นการแจกจ่ายงานทีละงานไปยังเครื่องที่ว่างอยู่เท่านั้น